HDOC Day-71

Dynamic Matrix Operations Using Pointer Notation

Aim

Write a C program to dynamically allocate two matrices and perform addition, subtraction, and multiplication using pointer notation. Implement each operation as a separate function and release all allocated memory properly.

Algorithm / Pseudocode

1. Read rows and columns of Matrix A and Matrix B.
2. Allocate memory dynamically for both matrices using malloc().
3. Read matrix elements using pointer notation.
4. If both matrices have equal dimensions, perform addition and subtraction.
5. If columns of A equal rows of B, perform multiplication.
6. Display the resulting matrices.
7. Release all dynamically allocated memory using free().

C Program

```
#include <stdio.h>
#include <stdlib.h>

int **allocateMatrix(int rows, int cols)
{
    int **m = malloc(rows * sizeof(int *));

    if (m == NULL)
        return NULL;

    for (int i = 0; i < rows; i++)
    {
        *(m + i) = malloc(cols * sizeof(int));

        if (*(m + i) == NULL)
        {
            for (int k = 0; k < i; k++)
                free(*(m + k));

            free(m);
            return NULL;
        }
    }

    return m;
}

void inputMatrix(int **m, int rows, int cols)
{
    for (int i = 0; i < rows; i++)
        for (int j = 0; j < cols; j++)
            scanf("%d", (*(m + i) + j));
}

void displayMatrix(int **m, int rows, int cols)
{
    for (int i = 0; i < rows; i++)
    {
        for (int j = 0; j < cols; j++)
            printf("%d\t", *(*(m + i) + j));

        printf("\n");
    }
```

```
}
```

C Program (continued)

```
void addMatrices(int **a, int **b, int **result,
                 int rows, int cols)
{
    for (int i = 0; i < rows; i++)
        for (int j = 0; j < cols; j++)
            *(*(result + i) + j) =
                *(*(a + i) + j) + *(*(b + i) + j);
}

void subtractMatrices(int **a, int **b, int **result,
                      int rows, int cols)
{
    for (int i = 0; i < rows; i++)
        for (int j = 0; j < cols; j++)
            *(*(result + i) + j) =
                *(*(a + i) + j) - *(*(b + i) + j);
}

void multiplyMatrices(int **a, int **b, int **result,
                      int r1, int c1, int c2)
{
    for (int i = 0; i < r1; i++)
    {
        for (int j = 0; j < c2; j++)
        {
            *(*(result + i) + j) = 0;

            for (int k = 0; k < c1; k++)
                *(*(result + i) + j) +=
                    *(*(a + i) + k) * *(*(b + k) + j);
        }
    }
}

void freeMatrix(int **m, int rows)
{
    for (int i = 0; i < rows; i++)
        free(*(m + i));

    free(m);
}
```

C Program (continued)

```
int main(void)
{
    int r1, c1, r2, c2;

    printf("Enter rows and columns of Matrix A: ");
    scanf("%d %d", &r1, &c1);

    printf("Enter rows and columns of Matrix B: ");
    scanf("%d %d", &r2, &c2);

    int **A = allocateMatrix(r1, c1);
    int **B = allocateMatrix(r2, c2);

    if (A == NULL || B == NULL)
    {
        printf("Memory allocation failed.\n");
        return 1;
    }

    printf("Enter elements of Matrix A:\n");
    inputMatrix(A, r1, c1);

    printf("Enter elements of Matrix B:\n");
    inputMatrix(B, r2, c2);

    if (r1 == r2 && c1 == c2)
    {
        int **R = allocateMatrix(r1, c1);

        addMatrices(A, B, R, r1, c1);
        printf("\nAddition:\n");
        displayMatrix(R, r1, c1);

        subtractMatrices(A, B, R, r1, c1);
        printf("\nSubtraction (A - B):\n");
        displayMatrix(R, r1, c1);

        freeMatrix(R, r1);
    }
    else
        printf("\nAddition and subtraction are not possible.\n");

    if (c1 == r2)
    {
        int **R = allocateMatrix(r1, c2);

        multiplyMatrices(A, B, R, r1, c1, c2);
        printf("\nMultiplication:\n");
        displayMatrix(R, r1, c2);

        freeMatrix(R, r1);
    }
    else
        printf("\nMultiplication is not possible.\n");

    freeMatrix(A, r1);
    freeMatrix(B, r2);

    return 0;
}
```

Test Cases

Test	Matrix A	Matrix B	Expected Result
1	2 x 2 1 2 3 4	2 x 2 5 6 7 8	Addition: 6 8 / 10 12 Subtraction: -4 -4 / -4 -4 Multiplication: 19 22 / 43 50
2	2 x 3 1 2 3 4 5 6	3 x 2 7 8 9 10 11 12	Addition/Subtraction: Not possible Multiplication: 58 64 / 139 154

Kali Linux Terminal Snippet - Test Case 1

```
┌──(kali㉿kali)-[~/Desktop/demo1]
└─$
gcc hdoc71.c
./a.out
Enter rows and columns of Matrix A: 2 2
Enter rows and columns of Matrix B: 2 2
Enter elements of Matrix A:
1 2
3 4
Enter elements of Matrix B:
5 6
7 8

Addition:
6       8
10      12

Subtraction (A - B):
-4      -4
-4      -4

Multiplication:
19      22
43      50
```

Kali Linux Terminal Snippet - Test Case 2

```
┌──(kali㉿kali)-[~/Desktop/demo1]
└─$
```

```
gcc hdoc71.c
./a.out
Enter rows and columns of Matrix A: 2 3
Enter rows and columns of Matrix B: 3 2
Enter elements of Matrix A:
1 2 3
4 5 6
Enter elements of Matrix B:
7 8
9 10
11 12

Addition and subtraction are not possible.

Multiplication:
58      64
139     154
```

Result

The program successfully performs valid matrix addition, subtraction and multiplication using dynamically allocated memory and pointer notation, and releases all allocated memory properly.